



INSTRUÇÕES DE ENTREGA

Data-limite: 29 / 05 / 2026

Plataforma: GitHub (repositório pessoal)

Pasta de destino: PEM-20261-Semana14/

Nomenclatura: ex01.c, ex02.c ... ex16.c

Obs.: Cada arquivo deve compilar sem erros com `gcc -Wall`

T1 Variáveis, Tipos e Operadores

Em C, a declaração precisa de uma variável não é mero protocolo — é um contrato com o compilador. Cada tipo primitivo (**int**, **float**, **double**, **char**) determina o tamanho em memória, o conjunto de valores representáveis e as operações válidas. Dominar esses tipos, a precedência de operadores e as conversões implícitas é o alicerce de toda lógica confiável.

01

Calculadora de Média Ponderada

Declare variáveis do tipo adequado para armazenar três notas (`double`) e seus respectivos pesos (`int`). Leia os valores via `scanf` e calcule a média ponderada:

$$\text{média} = (n1 \times p1 + n2 \times p2 + n3 \times p3) / (p1 + p2 + p3)$$

Imprima o resultado com **duas casas decimais**. Mostre também o tamanho em bytes de cada tipo utilizado, usando `sizeof`.

02

Conversor de Unidades com Overflow Controlado

Declare uma variável `int` para armazenar uma distância em centímetros (máx. 2.147.483.647). Leia o valor e converta-o para metros (`float`), quilômetros (`double`) e milhas (`double`), onde `1 milha = 1,60934 km`.

Desafio: Antes de realizar os cálculos, verifique se o valor informado é positivo. Caso contrário, exiba uma mensagem de erro sem encerrar o programa abruptamente. Justifique no código, via comentário, a escolha dos tipos de dados utilizados.

T2 Estruturas de Decisão — if / if encadeado

Todo programa não-trivial precisa responder ao contexto: bifurcar, escolher caminhos, reagir a exceções. O **if** e suas formas encadeadas (*if-else-if*) são a base do raciocínio condicional em C. Entender a avaliação de expressões booleanas, a precedência de operadores lógicos e os riscos do *dangling else* transforma o programador de um seguidor de sintaxe em um arquiteto de lógica.

03

Classificador de IMC

Leia o peso (kg) e a altura (m) de uma pessoa. Calcule o IMC (`peso / altura2`) e classifique utilizando **if encadeado**:

- $IMC < 18,5 \rightarrow$ Abaixo do peso
- $18,5 \leq IMC < 25,0 \rightarrow$ Peso normal
- $25,0 \leq IMC < 30,0 \rightarrow$ Sobrepeso
- $IMC \geq 30,0 \rightarrow$ Obesidade

Exiba o IMC com duas casas decimais e a classificação correspondente. Trate entradas inválidas (peso ou altura ≤ 0).

04

Validador de Triângulo e Classificação

Leia três valores reais que representam os lados de um possível triângulo. Utilize **if encadeado** para:

1. Verificar se os valores formam um triângulo válido (desigualdade triangular).
2. Se válido, classificar quanto aos *lados*: Equilátero, Isósceles ou Escaleno.
3. Se válido, classificar quanto aos *ângulos*: Retângulo, Obtusângulo ou Acutângulo — usando a relação entre os quadrados dos lados.

Cada decisão deve ser expressa como um bloco `if-else-if` claramente justificado com comentários.

T3 Estruturas de Repetição

Loops são o motor das iterações: sem eles, escreveríamos mil linhas para processar mil registros. C oferece três formas — **for**, **while** e **do-while** —, cada uma com semântica própria. A escolha consciente entre elas revela se o programador pensa em *quantas vezes* (for), em *enquanto uma condição* (while) ou em *ao menos uma vez* (do-while). Dominar laços aninhados e evitar loops infinitos acidentais é sinal de maturidade técnica.

05

Tabuada Interativa com Controle de Fluxo

Usando **laços aninhados**, gere e exiba a tabuada completa (de 1 a 10) para todos os números de 1 a 10, formatada em colunas alinhadas. O laço externo deve ser `for` e o interno, `while`.

Extensão: Ao final de cada tabuada, pergunte ao usuário se deseja continuar ('S'/'N'). Utilize `do-while` para controlar essa interação. O programa só termina quando o usuário digitar `'N'`.

06

Sequência de Collatz

A Conjectura de Collatz afirma que, para qualquer inteiro positivo `n`, a sequência abaixo sempre chega a 1:

- Se `n` for par: $n = n / 2$
- Se `n` for ímpar: $n = 3 \times n + 1$

Leia um inteiro positivo e, usando `while`, imprima cada termo da sequência até atingir 1. Contabilize e exiba o número de passos necessários. Implemente controle de entrada para rejeitar valores ≤ 0 .

T4 Vetores Unidimensionais (Arrays 1D)

Um vetor em C é uma sequência contígua de elementos do mesmo tipo, acessada por índices inteiros. É a estrutura de dados mais direta do paradigma imperativo e a porta de entrada para compreender como dados são dispostos na memória. A manipulação eficiente de vetores — preenchimento, busca, ordenação e análise estatística — é habilidade insubstituível em qualquer domínio de aplicação.

07 Estatísticas de um Conjunto de Dados

Declare um vetor de 10 elementos `float`. Leia os valores e calcule, em uma única passagem pelo vetor (loop único):

- Média aritmética
- Maior e menor valor
- Soma dos valores acima da média

Exiba os resultados formatados. **Proibido** usar bibliotecas matemáticas externas — apenas aritmética básica com o vetor.

08 Análise e Manipulação de Vetores

Leia um vetor de `N` inteiros (`N` definido por `#define`, valor sugerido: 8).

Exiba o vetor original e, em seguida:

- Informe o maior valor armazenado e o índice onde ele se encontra;
- Informe o menor valor armazenado e o índice onde ele se encontra;
- Conte quantos elementos do vetor são pares e quantos são ímpares;
- Gere um segundo vetor contendo os mesmos elementos do vetor original, porém em ordem inversa;

Por fim, leia um valor fornecido pelo usuário e realize uma **busca linear** no vetor, informando o índice onde o valor foi encontrado ou uma mensagem indicando que ele não existe no vetor.

T5 Matrizes Multidimensionais (Arrays 2D)

Quando uma dimensão não é suficiente para modelar a realidade — tabelas, grades, imagens, grafos de adjacência —, as matrizes bidimensionais entram em cena. Em C, uma matriz é um vetor de vetores, armazenada linha a linha (*row-major order*). Entender como os índices mapeiam endereços de memória é essencial tanto para a correção do algoritmo quanto para a eficiência do acesso ao cache.

09

Multiplicação de Matrizes

Declare duas matrizes 3×3 de inteiros. Leia seus valores e calcule o produto matricial **$C = A \times B$** , onde:

$$C[i][j] = \sum A[i][k] \times B[k][j]$$

Exiba as três matrizes formatadas (A, B e C) com colunas alinhadas. Adicione comentários explicando a correspondência entre os índices do loop triplo e a definição algébrica.

10

Jogo da Memória — Análise de Tabuleiro

Represente um tabuleiro 4×4 como uma matriz de inteiros, onde cada valor indica um símbolo de carta (1 a 8, cada símbolo aparece exatamente duas vezes). Preencha a matriz manualmente no código (hardcoded) ou por leitura.

Implemente funções que:

- Exibam o tabuleiro formatado como grade.
- Verifiquem se todos os pares estão em posições simétricas em relação à diagonal principal.
- Calculem a diagonal principal e a diagonal secundária da matriz.

T6

Ponteiros — Conceito, Operadores, Aritmética, Arrays e Funções

Ponteiros são o que distingue C de linguagens de alto nível: eles expõem o endereço de memória como um valor manipulável. O operador de referência `&` obtém o endereço de uma variável; o operador de desreferenciação `*` acessa o valor no endereço apontado. A aritmética de ponteiros permite navegar estruturas contíguas com precisão cirúrgica. Compreender ponteiros é compreender *onde* os dados vivem — e isso muda tudo.

11 Aritmética de Ponteiros sobre um Vetor

Declare um vetor de 6 inteiros e um ponteiro `int *p` inicializado com o endereço do primeiro elemento. Sem usar notação de índice (`[]`), percorra todo o vetor usando **somente aritmética de ponteiros** (`p++`, `*(p+i)`) para:

- Imprimir cada elemento e seu endereço de memória (em hexadecimal via `%p`).
- Calcular a soma dos elementos.
- Dobrar o valor de cada elemento *in-place*, via desreferenciação.

Imprima os endereços e confirme que a diferença entre endereços consecutivos equivale a `sizeof(int)`.

12 Funções com Passagem por Ponteiro

Implemente as seguintes funções, todas recebendo parâmetros **exclusivamente por ponteiro**:

- `void troca(int *a, int *b)` — troca os valores de duas variáveis.
- `void minMax(int *v, int n, int *min, int *max)` — retorna o menor e o maior elemento de um vetor de tamanho `n`.
- `void normalizaVetor(float *v, int n)` — divide cada elemento pelo maior valor do vetor, normalizando para o intervalo `[0,1]`.

Demonstre cada função em `main`, imprimindo os resultados antes e depois das chamadas para comprovar que as alterações via ponteiro afetaram as variáveis originais.

Programas crescem. Funções bem definidas transformam complexidade monolítica em unidades testáveis, reusáveis e legíveis. Em C, uma função possui protótipo (declaração), definição e chamada — cada uma com papel preciso no ciclo de compilação. A modularização bem aplicada reduz acoplamento, facilita manutenção e é o primeiro passo para uma arquitetura escalável, seja em sistemas embarcados ou em grandes bases de código.

13

Biblioteca de Operações com Strings

Implemente um módulo (arquivo `.c` com respectivo `.h`) contendo as funções:

- `int contaVogais(char *s)` — conta vogais (maiúsculas e minúsculas).
- `void inverteCString(char *s)` — inverte a string *in-place*.
- `int ePalindromo(char *s)` — retorna 1 se for palíndromo, 0 caso contrário.

Proibido usar funções de `string.h` exceto `strlen`. O arquivo `main.c` deve incluir o `.h` e testar cada função com ao menos dois casos. Compile com: `gcc main.c stringutil.c -o prog`.

14

Calculadora Modular com Menu

Crie um programa com menu interativo que execute operações matemáticas. Cada operação deve ser implementada em uma **função separada**, todas com protótipos declarados antes de `main`:

- Máximo Divisor Comum — algoritmo de Euclides.
- Potência inteira — sem usar `pow()`.
- Verificador de número primo.
- Conversão decimal → binário (retorna string).

O menu deve ser controlado por `do-while` com opção de saída. Cada função deve ter documentação em comentário de bloco descrevendo: parâmetros, retorno e complexidade algorítmica.

Recursividade é a técnica em que uma função resolve um problema chamando a si mesma com uma entrada reduzida, até atingir um **caso base** que não requer nova chamada. Mais do que uma curiosidade teórica, é a forma natural de expressar algoritmos sobre estruturas auto-similares: árvores, divisões sucessivas, problemas combinatórios. Todo programador deve saber quando a recursão é elegante — e quando a iteração é mais eficiente. O equilíbrio entre pilha de chamadas e clareza de código é uma decisão de engenharia.

15

Torre de Hanói — Solução Recursiva Comentada

Implemente recursivamente a solução da **Torre de Hanói** para N discos (leia N do usuário, $1 \leq N \leq 10$).

A função `void hanoi(int n, char origem, char destino, char auxiliar)` deve imprimir cada movimento no formato: `Mova disco X de [origem] para [destino]`.

Análise obrigatória: Ao final, exiba o número total de movimentos realizados e verifique se coincide com $2^n - 1$. Adicione comentários identificando o *caso base* e o *passo recursivo*.

16

Busca Recursiva e Soma por Divisão do Vetor

Implemente duas funções de forma **estritamente recursiva**:

- `int buscaRec(int *v, int n, int chave)` — percorre recursivamente o vetor e retorna o índice da primeira ocorrência do elemento procurado ou `-1` caso não exista.
- `int somaRec(int *v, int ini, int fim)` — calcula a soma dos elementos do vetor utilizando a estratégia de *divisão sucessiva* do intervalo:
 - divida o vetor em duas partes;
 - some recursivamente cada metade;
 - retorne a soma total.

Gere um vetor de 12 inteiros aleatórios (utilize `rand()`) e exiba todos os valores armazenados.

Em seguida:

- Calcule e exiba a soma total do vetor usando `somaRec`;
- Leia um valor informado pelo usuário;
- Utilize `buscaRec` para localizar o valor no vetor;
- Informe o índice encontrado ou uma mensagem indicando que o valor não existe.

Durante a execução, exiba também:

- a quantidade total de chamadas recursivas realizadas em cada função;
- a profundidade máxima de recursão atingida.

Obrigatório: adicionar comentários identificando claramente o *caso base* e o *passo recursivo* de cada função.